

CSE548 Fall 2023 Midterm 1 Solutions

October 2023

Question 1

(a)

Initiate: Build an array d of length $2n$. Set $d[i] = 0$ for $i = 1 \rightarrow 2n$.
for $i = 1 \rightarrow n_\ell$ **do**
 for $j = 1 \rightarrow n_r$ **do**
 $x =$ the length of the i th subtile in $\mathcal{L}$;
 $y =$ the length of the j th subtile in $\mathcal{R}$;
 $d[x + y] += (x + y)$;
 end for
end for
Output d .

3 points for the double-level loop.

1 point for $d[x + y] += (x + y)$.

(b) Build two polynomials: $y_{\mathcal{L}} = \sum_{i=1}^n (c_{\mathcal{L},i} x^i)$ and $y_{\mathcal{R}} = \sum_{i=1}^n (c_{\mathcal{R},i} x^i)$.

The coefficient $c_{\mathcal{L},i} = 1$ if $L_i \in \mathcal{L}$ and 0 otherwise.

The coefficient $c_{\mathcal{R},i} = 1$ if $R_i \in \mathcal{R}$ and 0 otherwise. **5 points.**

Use FFT to calculate $y_{\mathcal{L}} y_{\mathcal{R}} = \sum_{i=1}^{2n} (c_i x^i)$. **5 points.**

Output $d_i = i \cdot c_i$ for $i = 1 \rightarrow 2n$. **5 points.**

(c) *There is a problem in 1(c). The approach outlined in the hint does not work when the number of copies of each kind of subtile is an arbitrary positive integer. It works, e.g., when those numbers are restricted to powers of 2.* **2 points for all students who have touched because of this problem.**

Suppose L_k has $c_{\mathcal{L},k}$ copies and R_k has $c_{\mathcal{R},k}$ copies.

Create $1 + \log m$ polynomials for the left set $L(i) = \sum_k x^k$ for all k where $c_{\mathcal{L},k} = 2^i$ for $i = 0$ to $\log m$. Do the same for the right set ($R(i)$ for $i = 0$ to $\log m$). **1 point.**

The result should be:

$$\sum_{i=0}^{\log m} \left\{ \sum_{j=0}^{\log m} \{ \min(2^i, 2^j) L(i) R(j) \} \right\} = \sum_{i=1}^{2n} (c_i x^i). \textbf{2 points.}$$

Output $d_i = i \cdot c_i$ for $i = 1 \rightarrow 2n$. **1 point.**

Students may get some points if they gave some incomplete approaches,

Question 2

Karatsuba's algorithm multiplies two n -bit integers in $\Theta(n^{\log_2 3})$ time. This problem asks you to use Karatsuba's algorithm to multiply $m \geq 2$ binary integers each of which is n bits long.

Input: binary integers $x_1, x_2, \dots, x_m$ each exactly n bits long
Output: product $x_1 x_2 \dots x_m$
Algorithm:

1. $z \leftarrow x_1$
2. **for** $i \leftarrow 2$ **to** m **do**
3. $t \leftarrow z \times x_i$ computed using Karatsuba's algorithm
4. $z \leftarrow t$
5. **return** z

Figure 1: [Question ??] Naïvely computing the product of m binary numbers of length n each.

- (a) [6 Points] Show that the algorithm given in Figure 1 takes $\Theta(m(mn)^{\log_2 3})$ time to multiply $m \geq 2$ binary integers containing n bits each.

Solution: Calculating the product of two n -bit integers using Karatsuba's algorithm takes $\Theta(n^{\log_2 3})$ time.

Upper bound. Line 3 has increasing cost because as i increases the number of bits in z also increases. In the worst case, at each iteration i , the number of bits of z increased by n .

i	number of bits in z
2	n
3	$n + n = 2n$
4	$n + 2n = 3n$
-	-
m	$(m-1)n$

So, executing line 3 takes $\mathcal{O}((mn)^{\log_2 3})$ time. This line is executed $m-1$ times. Hence, the overall running time is $\mathcal{O}(m(mn)^{\log_2 3})$.

Lower bound. Let τ_i be the time it takes to execute the i -th ($2 \leq i \leq m$) iteration of the loop in line 2, i.e., to compute $z \times x_i$ in line 3.

Then the total time to execute all iterations of the loop in line 2 is $\sum_{i=2}^m \tau_i$.

Now, $\sum_{i=2}^m \tau_i \geq \sum_{i=m/2}^m \tau_i \geq \sum_{i=m/2}^m \tau_{m/2} = \tau_{m/2} (\sum_{i=m/2}^m 1) = \Theta(m(\tau_{m/2})) = \Theta(m((m/2)n)^{\log_2 3}) = \Theta(m((mn)^{\log_2 3}))$

Hence, the algorithm takes $\Omega(m((mn)^{\log_2 3}))$ time execute.

Tight bound. By combining the upper and lower bounds, we conclude that the algorithm takes $\Theta(m((mn)^{\log_2 3}))$ time to run.

Finding the upper bound 3 points

Finding the lower bound 3 points

- (b) [9 Points] Give a recursive divide-and-conquer algorithm that runs in $\Theta(m(mn)^{\log_2 3})$ time to multiply $m \geq 2$ binary integers each of which is n bits long. You must use Karatsuba's algorithm whenever multiplying two integers. Write down the recurrence relation describing the running time of the algorithm and solve it.

Solution: Split the m binary integers into two sets of equal size (in fact, $\lceil \frac{m}{2} \rceil$ and $\lfloor \frac{m}{2} \rfloor$) and find the product of each set recursively. Then multiply the two products using Karatsuba's algorithm. In the base case, we directly multiply two integers consisting of ≤ 1 bits each in $\Theta(1)$ time.

Recurrence relation (since we can drop floors and ceilings in Master Theorem):

$$T(m) = \begin{cases} \Theta(1), & \text{if } m \leq 1, \\ 2T\left(\frac{m}{2}\right) + \Theta((mn)^{\log_2 3}), & \text{otherwise.}, \end{cases}$$

We then apply the third case of the Master Theorem to get the required bound.

Identifying and explaining the algorithm 5 points

Finding the recurrence relation 3 points

Solving the recurrence 1 point

Question 3

This problem asks you to use Akra-Bazzi to solve ugly recurrences of the following form¹.

$$T(n) = \begin{cases} \Theta(1), & \text{if } 2 \leq n \leq n_0, \\ \sum_{i=1}^k a_i n^{\alpha(1-b_i)} T(n^{b_i}) + \Theta(n^\alpha (\log n)^\beta), & \text{otherwise,} \end{cases}$$

where, $k \geq 1$ is an integer constant; $a_i > 0$ and $b_i \in (0, 1)$ are constants for $1 \leq i \leq k$; $n \geq 2$ is a real number; α and β are real constants; $n_0 \geq 2$ is a constant and $n_0 \geq 2^{\max\{\frac{1}{b_i}, \frac{1}{1-b_i}\}}$ for $1 \leq i \leq k$. Let p be the unique real number for which $\sum_{i=1}^k a_i b_i^p = 1$.

(a) Suppose $T'(n) = \frac{T(n)}{n^\alpha}$. Show that

$$T'(n) = \begin{cases} \Theta(1), & \text{if } 2 \leq n \leq n_0, \\ \sum_{i=1}^k a_i T'(n^{b_i}) + \Theta((\log n)^\beta), & \text{otherwise.} \end{cases}$$

Solution

The n_0 bound on the constant case stays the same.

Otherwise, we have

$$T'(n) = \sum_{i=1}^k a_i \frac{1}{n^\alpha} n^{\alpha(1-b_i)} T(n^{b_i}) + \frac{1}{n^\alpha} \Theta(n^\alpha (\log n)^\beta) = \sum_{i=1}^k a_i \frac{T(n^{b_i})}{n^{b_i \alpha}} + \frac{1}{n^\alpha} \Theta(n^\alpha (\log n)^\beta) = \sum_{i=1}^k a_i T'(n^{b_i}) + \Theta((\log n)^\beta)$$

$$\text{Note: } T'(n^{b_i}) = \frac{T(n^{b_i})}{(n^{b_i})^\alpha}$$

$$T'(n) = \sum_{i=1}^k a_i T'(n^{b_i}) + \Theta((\log n)^\beta)$$

For the other case, as desired.

Grading

- 0.5 points for $\Theta(1)$ bounds.
- 1.5 points for the “otherwise” condition.
- 0.5 points of this for correctly identifying $T'(n^{b_i}) = \frac{T(n^{b_i})}{(n^{b_i})^\alpha}$

(b) Suppose $n = 2^x$, $n_0 = 2^{x_0}$, and $T''(x) = T'(2^x)$. Show that

$$T''(x) = \begin{cases} \Theta(1), & \text{if } 1 \leq x \leq x_0, \\ \sum_{i=1}^k a_i T''(b_i x) + \Theta(x^\beta), & \text{otherwise.} \end{cases}$$

Solution

We have $n = 2^x$.

At $n = 2$, $x = 1$, and we also have $n_0 = 2^{x_0}$, therefore our bounds for the constant case on $T'(x)$ are now $2 \leq 2^x \leq 2^{x_0}$.

For $T''(x)$, the upper bound is transformed to x_0 , and the lower bound becomes 1.

For the otherwise case,

$$T'(2^x) = \sum_{i=1}^k a_i T'(2^{b_i x}) + \Theta((\log 2^x)^\beta) = \sum_{i=1}^k a_i T'(2^{b_i x}) + \Theta((x)^\beta)$$

When 2^x is replaced with x for $T''(x)$, we have

$$T''(x) = \sum_{i=1}^k a_i T''(b_i x) + \Theta((x)^\beta)$$

As desired.

¹Recurrences of this form appear in the analysis of running times of several FFT variants, column sort, etc.

Grading

- 0.5 points for $\Theta(1)$ bounds.
- 1.5 points for the “otherwise” condition.
 - 0.5 points of this for correctly evaluating the $\Theta((x)^\beta)$

(c) Use the Akra-Bazzi formula to show that the recurrence from part (b) has the following solutions:

$$T''(x) = \begin{cases} \Theta(x^\beta \log x), & \text{if } p = \beta, \\ \Theta\left(\left(1 - \frac{1}{\beta-p}\right)x^p + \left(\frac{1}{\beta-p}\right)x^\beta\right), & \text{if } p \neq \beta. \end{cases}$$

Solution

We meet all Akra-Bazzi criteria. The first four are stated directly in part (a), and the last two we can assume.

Here, $g(x) = x^\beta$.

$$\int_1^x \frac{g(u)}{u^{p+1}} du = \int_1^x \frac{u^\beta}{u^{p+1}} du = \int_1^x u^{\beta-(p+1)} du$$

As the hint specifies, this integral has different results depending on the value of the exponent.

If $p = \beta$, then we have

$$\int_1^x u^{\beta-(p+1)} du = \int_1^x u^{-1} du = \ln u \Big|_1^x = \ln x - \ln 1 = \ln x$$

Therefore, the time complexity in this case is

$$T''(x) = \Theta(x^p (1 + \ln x)) = \Theta(x^\beta (1 + \ln x)) = \Theta(x^\beta \ln x)$$

Both natural log and log with no base are acceptable solutions. The base in this case does not matter as long as the base is a constant.

If $p \neq \beta$

$$\int_1^x u^{\beta-p-1} du = \frac{u^{\beta-p}}{\beta-p} \Big|_1^x = \frac{x^{\beta-p}}{\beta-p} - \frac{1^{\beta-p}}{\beta-p} = \frac{x^{\beta-p}}{\beta-p} - \frac{1}{\beta-p}$$

Therefore, the complexity in this case is

$$\begin{aligned} T''(x) &= \Theta\left(x^p \left(1 + \frac{x^{\beta-p}}{\beta-p} - \frac{1}{\beta-p}\right)\right) = \Theta\left(x^p \left(1 - \frac{1}{\beta-p} + \frac{x^{\beta-p}}{\beta-p}\right)\right) = \Theta\left(x^p \left(1 - \frac{1}{\beta-p}\right) + x^p \frac{x^{\beta-p}}{\beta-p}\right) = \\ &= \Theta\left(\left(1 - \frac{1}{\beta-p}\right)x^p + \left(\frac{1}{\beta-p}\right)x^\beta\right) \end{aligned}$$

Grading

- 2 points for $p = \beta$ case
- 4 points for $p \neq \beta$ case

(d) Use the solution from part (c) to show that

$$T(n) = \begin{cases} \Theta(n^\alpha (\log n)^\beta), & \text{if } p < \beta, \\ \Theta(n^\alpha (\log n)^\beta \log \log n), & \text{if } p = \beta, \\ \Theta(n^\alpha (\log n)^p), & \text{if } p > \beta. \end{cases}$$

Solution

If $p = \beta$

We have $T''(x) = \Theta(x^\beta \log x)$

$$T'(n) = \Theta((\log n)^\beta \log \log n)$$

$$T(n) = \Theta(n^\alpha (\log n)^\beta \log \log n)$$

If $p < \beta$

$$\text{We have } T''(x) = \Theta\left(\left(1 - \frac{1}{\beta-p}\right)x^p + \left(\frac{1}{\beta-p}\right)x^\beta\right)$$

$$T'(n) = \Theta\left(\left(1 - \frac{1}{\beta-p}\right)(\log n)^p + \left(\frac{1}{\beta-p}\right)(\log n)^\beta\right)$$

If $p < \beta$, then the dominating term is $\left(\frac{1}{\beta-p}\right)(\log n)^\beta$, so

$$T'(n) = \Theta\left(\frac{1}{\beta-p}\right)(\log n)^\beta$$

$$T(n) = \Theta(n^\alpha \left(\frac{1}{\beta-p}\right)(\log n)^\beta)$$

$\frac{1}{\beta-p}$ is determined from constants and can be removed.

$$T(n) = \Theta(n^\alpha (\log n)^\beta)$$

If $p > \beta$

$$\text{We have } T''(x) = \Theta\left(\left(1 - \frac{1}{\beta-p}\right)x^p + \left(\frac{1}{\beta-p}\right)x^\beta\right)$$

$$T'(n) = \Theta\left(\left(1 - \frac{1}{\beta-p}\right)(\log n)^p + \left(\frac{1}{\beta-p}\right)(\log n)^\beta\right)$$

If $p < \beta$, then the dominating term is $\left(1 - \frac{1}{\beta-p}\right)(\log n)^p$, so

$$T'(n) = \Theta\left(\left(1 - \frac{1}{\beta-p}\right)(\log n)^p\right)$$

$$T(n) = \Theta(n^\alpha \left(1 - \frac{1}{\beta-p}\right)(\log n)^p)$$

$\frac{1}{\beta-p}$ is determined from constants and can be removed.

$$T(n) = \Theta(n^\alpha (\log n)^p)$$

Grading

- 2 points per case
- 1 point total was allocated for explaining dominating terms.
- No points were removed for keeping the constant coefficients.

(e) Now use the solution from part (d) to solve the following recurrences:

$$T_1(n) = \begin{cases} \Theta(1), & \text{if } 2 \leq n \leq n_0, \\ n^{\frac{1}{2}}T_1\left(n^{\frac{1}{2}}\right) + \Theta(n), & \text{otherwise,} \end{cases}$$

$$T_2(n) = \begin{cases} \Theta(1), & \text{if } 2 \leq n \leq n_0, \\ n^{\frac{1}{3}}T_2\left(n^{\frac{2}{3}}\right) + n^{\frac{2}{3}}T_2\left(n^{\frac{1}{3}}\right) + \Theta(n \log n), & \text{otherwise,} \end{cases}$$

$$T_3(n) = \begin{cases} \Theta(1), & \text{if } 2 \leq n \leq n_0, \\ 2n^{\frac{15}{8}}T_3\left(n^{\frac{1}{16}}\right) + n^{\frac{3}{2}}T_3\left(n^{\frac{1}{4}}\right) + \Theta(n^2 \log n), & \text{otherwise,} \end{cases}$$

Solution

For $T_1(n)$:

$$\alpha = 1, \beta = 0$$

$$a_1 = 1, b_1 = \frac{1}{2}$$

$$p = 0$$

$p = \beta$ case

$$T_1(n) = \Theta(n^\alpha (\log n)^\beta \log \log n) = \Theta(n \log \log n)$$

For $T_2(n)$:

$$\alpha = 1, \beta = 1$$

$$a_1 = 1, b_1 = \frac{2}{3}, a_2 = 1, b_2 = \frac{1}{3}$$

$$p = 1$$

$p = \beta$ case

$$\begin{vmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{vmatrix} \otimes \begin{vmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{vmatrix} = \begin{vmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{vmatrix}$$

Figure 2: [Question (b)] Bitwise product of two 3×3 bit matrices.

$$T_2(n) = \Theta(n^\alpha (\log n)^\beta \log \log n) = \Theta(n (\log n) \log \log n)$$

For $T_3(x)$:

$$\alpha = 2, \beta = 1$$

$$a_1 = 2, b_1 = \frac{1}{16}, a_2 = 1, b_2 = \frac{1}{4}$$

$$p = \frac{1}{2}$$

$p < \beta$ case

$$T_3(n) = \Theta(n^\alpha (\log n)^\beta) = \Theta(n^2 (\log n))$$

Grading

- 3 points per case
 - 1 point for finding α , β , and a_i/b_i s
 - 1 point for finding p
 - 1 point for choosing correct case and substituting correctly

Question 4

Bitwise Matrix Multiplication. Suppose for some positive integer n , you are given two $n \times n$ matrices X and Y in which every entry is a single bit (either 0 or 1). Therefore, each matrix occupies exactly n^2 bits. You multiply X and Y using bitwise OR ($\vee$) and bitwise AND ($\wedge$) operators only. You end up with an $n \times n$ product matrix Z in which each entry is a single bit, and for $1 \leq i, j \leq n$, entry $z_{i,j}$ of Z is defined as follows, where $x_{i,k}$'s and $y_{k,j}$'s are entries of X and Y , respectively.

$$\begin{aligned} z_{i,j} &= \bigvee_{k=1}^n (x_{i,k} \wedge y_{k,j}) \\ &= (x_{i,1} \wedge y_{1,j}) \vee (x_{i,2} \wedge y_{2,j}) \vee (x_{i,3} \wedge y_{3,j}) \vee \dots \vee (x_{i,n} \wedge y_{n,j}) \end{aligned}$$

This product is similar to the product in the standard matrix multiplication algorithm we saw in the class, except that we have replaced the ' $\times$ ' and ' $+$ ' operators with ' $\wedge$ ' and ' $\vee$ ' operators, respectively. Clearly, all entries of Z can be computed in $\Theta(n^3)$ time using a naïve looping code.

Figure 2 shows an example, where we use the $\otimes$ operator to indicate that this is not standard matrix multiplication.

Now, answer the following questions.

- (a) [**3 Points**] It turns out that the standard $\Theta(n^3)$ time recursive matrix multiplication algorithm that we saw in the class can be easily modified (by replacing ' $\times$ ' and ' $+$ ' with ' $\wedge$ ' and ' $\vee$ ', respectively) to correctly compute the bitwise product of X and Y as defined above using only $\Theta(n^2)$ bits of space. However, Strassen's algorithm cannot be used to compute Z in $\Theta(n^2)$ bits of space using those bitwise operators. Why?

Solution: We can not use Strassen's algorithm because it relies on properties of subtraction, but in this domain, subtraction is not defined.

Identifying that subtraction operation is the problem 1 point

Explaining why subtraction is the problem 2 points

- (b) [7 Points] Suppose I allow you to use $\Theta(n^2 \log n)$ bits of space. Now, can you use Strassen's algorithm without replacing the standard ' $\times$ ' and ' $+$ ' operators to compute Z correctly? How?

Solution: Yes, we can. If we allocate $1 + \lceil \log_2 n \rceil$ bits for each cell of the matrix, we can store any nonnegative integer of value up to n in it. So, then we can perform matrix multiplication with Strassen's algorithm without replacing the standard ' $\times$ ' and ' $+$ ' operators to compute Z . Each cell of Z either contains a 0 or a value ≥ 1 . Now, we iterate through matrix Z and replace every entry with value > 1 with a 1. That will give us the required matrix.

Answering either Strassen's algorithm can be used or not 1 point

Explaining how Strassen's can be used to calculate Z 6 points